

Playwright with JavaScript – Complete Interview Preparation Guide

1. What is Playwright?

Definition

Playwright is a modern open-source automation framework developed by Microsoft for:

- UI Automation
- End-to-End Testing
- API Testing
- Cross-browser Testing

It supports:

- Chromium
 - Firefox
 - WebKit
-

2. Why Playwright is Popular?

Advantages

- Faster execution
 - Auto-waiting
 - Cross-browser support
 - Parallel execution
 - Network interception
 - API testing support
 - Mobile emulation
 - Built-in assertions
 - Multi-tab handling
 - Headless execution
-

3. Playwright Architecture

```
Test Scripts
  ↓
Playwright Client Library
  ↓
Browser Server
  ↓
Browser Engine
  ↓
Web Application
```

4. Playwright vs Selenium

Playwright	Selenium
Modern architecture	Old architecture
Faster	Comparatively slower
Auto waits	Explicit waits needed
Built-in network handling	External libraries required
Multiple tabs easy	Complex handling
Supports API testing	Limited API support
Better parallel execution	Requires grid setup

5. Installation Commands

Install Node.js

Required runtime environment.

Initialize Project

```
npm init -y
```

Install Playwright

```
npm init playwright@latest
```

6. Playwright Project Structure

```
project
├──
├── tests
├── pages
├── utils
├── fixtures
├── playwright.config.js
├── package.json
└── reports
```

7. Playwright Test Example

```
const { test, expect } = require('@playwright/test');

test('Login Test', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

8. Important Playwright Components

Component	Purpose
Browser	Browser instance
Context	Isolated browser session
Page	Single tab/page
Locator	Element locator

Component	Purpose
Test Runner	Executes tests

9. What is Browser Context?

Browser Context is an isolated browser session.

Benefits:

- Independent cookies
- Independent cache
- Parallel execution support
- Multi-user testing

10. Browser Launch Example

```
const { chromium } = require('playwright');

(async () => {
  const browser = await chromium.launch({ headless: false });
  const page = await browser.newPage();
  await page.goto('https://example.com');
  await browser.close();
})();
```

11. Common Locator Strategies

Locator	Example
id	page.locator('#id')
class	page.locator('.class')
text	page.getByText()
role	page.getByRole()
xpath	page.locator('//button')
css	page.locator('button')

12. Recommended Locator Priority

1. getByRole()
2. getByLabel()
3. getByPlaceholder()
4. getByText()
5. CSS/XPath

13. Auto-Waiting in Playwright

Playwright automatically waits for:

- Elements visible
- Elements stable
- Elements enabled
- Page load

This reduces flaky tests.

14. Assertions in Playwright

Common Assertions

```
await expect(page).toHaveTitle('Example');  
await expect(locator).toBeVisible();  
await expect(locator).toContainText('Welcome');
```

15. Types of Waits

Auto Wait

Built-in wait.

Explicit Wait

```
await page.waitForTimeout(3000);
```

Wait for Selector

```
await page.waitForSelector('#login');
```

Wait for Load State

```
await page.waitForLoadState('networkidle');
```

16. Handling Input Fields

```
await page.fill('#username', 'admin');
```

Alternative:

```
await page.locator('#username').fill('admin');
```

17. Click Operation

```
await page.click('#loginBtn');
```

Recommended:

```
await page.locator('#loginBtn').click();
```

18. Dropdown Handling

```
await page.selectOption('#country', 'India');
```

19. Checkbox Handling

```
await page.check('#remember');  
await page.uncheck('#remember');
```

20. Radio Button Handling

```
await page.check('#male');
```

21. Keyboard Actions

```
await page.keyboard.press('Enter');  
await page.keyboard.type('Playwright');
```

22. Mouse Actions

```
await page.hover('#menu');  
await page.dragAndDrop('#source', '#target');
```

23. Handling Alerts

```
page.on('dialog', async dialog => {  
  console.log(dialog.message());  
});
```

```
    await dialog.accept();  
  });
```

24. Handling Multiple Tabs

```
const [newPage] = await Promise.all([  
  context.waitForEvent('page'),  
  page.click('#open')  
]);
```

25. Handling Frames

```
const frame = page.frameLocator('#frame1');  
await frame.locator('#submit').click();
```

26. File Upload

```
await page.setInputFiles('#upload', 'test.pdf');
```

27. File Download

```
const downloadPromise = page.waitForEvent('download');  
await page.click('#download');  
const download = await downloadPromise;
```

28. Screenshots

```
await page.screenshot({ path: 'page.png' });
```


Element Screenshot:

```
await locator.screenshot({ path: 'element.png' });
```

29. Video Recording

Configured in:

```
use: {  
  video: 'on'  
}
```

30. Trace Viewer

Used for debugging failures.

Enable:

```
use: {  
  trace: 'on'  
}
```

View trace:

```
npx playwright show-trace trace.zip
```

31. Page Object Model (POM)

Why POM?

- Better maintainability
- Reusable code
- Cleaner framework
- Centralized locators

32. POM Example

LoginPage.js

```
class LoginPage {

  constructor(page) {
    this.page = page;
    this.username = '#username';
    this.password = '#password';
    this.loginBtn = '#login';
  }

  async login(user, pass) {
    await this.page.fill(this.username, user);
    await this.page.fill(this.password, pass);
    await this.page.click(this.loginBtn);
  }
}

module.exports = LoginPage;
```

33. Test Using POM

```
const { test } = require('@playwright/test');
const LoginPage = require('../pages/LoginPage');

test('Login', async ({ page }) => {
  const login = new LoginPage(page);
  await page.goto('https://example.com');
  await login.login('admin', 'admin123');
});
```

34. Fixtures in Playwright

Fixtures provide reusable setup.

Example:

```
const base = require('@playwright/test');

exports.test = base.test.extend({
  loginPage: async ({ page }, use) => {
    await use(page);
  }
});
```

35. Hooks in Playwright

```
test.beforeEach(async ({ page }) => {
  await page.goto('https://example.com');
});

test.afterEach(async ({ page }) => {
  console.log('Test Completed');
});
```

36. API Testing in Playwright

```
const response = await request.get('https://reqres.in/api/users');
expect(response.status()).toBe(200);
```

37. API POST Request Example

```
const response = await request.post('/users', {
  data: {
    name: 'Rakesh',
    job: 'QA'
  }
});
```

38. Network Interception

```
await page.route('**/api/**', async route => {  
  await route.continue();  
});
```

39. Mock API Response

```
await page.route('**/api/users', async route => {  
  await route.fulfill({  
    status: 200,  
    body: JSON.stringify({ success: true })  
  });  
});
```

40. Parallel Execution

Configured in:

```
workers: 4
```

Benefits:

- Faster execution
- Better CI/CD performance

41. Retry Mechanism

```
retries: 2
```

Useful for flaky tests.

42. Environment Configuration

Example:

```
use: {  
  baseUrl: 'https://staging.example.com'  
}
```

43. Playwright Configuration File

playwright.config.js

```
module.exports = {  
  timeout: 30000,  
  retries: 1,  
  use: {  
    headless: true,  
    screenshot: 'only-on-failure'  
  }  
};
```

44. Reporting in Playwright

HTML Report

```
npx playwright show-report
```

Configure Reporter

```
reporter: [['html']]
```

45. Cross Browser Execution

```
projects: [  
  { name: 'Chromium' },  
  { name: 'Firefox' },  
  { name: 'WebKit' }  
]
```

46. Headless vs Headed Mode

Headless	Headed
No UI	Browser visible
Faster	Useful for debugging
CI/CD friendly	Local debugging

47. Handling Dynamic Elements

Strategies:

- Stable locators
- Auto waits
- Relative locators
- Dynamic XPath carefully

48. Handling Shadow DOM

```
await page.locator('css=custom-element').locator('#shadowBtn').click();
```

49. Authentication Handling

Save storage state:

```
await context.storageState({ path: 'auth.json' });
```

Reuse:

```
storageState: 'auth.json'
```

50. Playwright Test Lifecycle

```
BeforeAll
  ↓
BeforeEach
  ↓
Test Execution
  ↓
AfterEach
  ↓
AfterAll
```

51. CI/CD Integration

Playwright integrates with:

- Jenkins
- GitHub Actions
- Azure DevOps
- GitLab CI

52. GitHub Actions Example

```
name: Playwright Tests

on: [push]

jobs:
  test:
    runs-on: ubuntu-latest
```

```
steps:
  - uses: actions/checkout@v3
  - uses: actions/setup-node@v3
  - run: npm install
  - run: npx playwright test
```

53. Common Playwright Interview Questions

Why Playwright over Selenium?

Answer:

- Faster execution
 - Auto waits
 - Better architecture
 - API support
 - Stable execution
-

What is Browser Context?

Answer:

- Isolated browser session
 - Helps parallel execution
 - Separate cookies/cache
-

Why Playwright is Faster?

Answer:

- Direct browser communication
 - WebSocket architecture
 - Auto waits
 - Less dependency layers
-

54. Real-Time Framework Questions

How do you design Playwright framework?

Answer:

- POM structure
 - Utility classes
 - Fixtures
 - Environment configs
 - Reporting integration
 - CI/CD support
-

How do you handle flaky tests?

Answer:

- Proper waits
 - Stable locators
 - Retry mechanism
 - Network synchronization
-

How do you execute tests in parallel?

Answer:

- Configure workers
 - Independent test data
 - Browser contexts
-

55. Playwright Best Practices

- Use locators instead of page methods
- Prefer getByRole()
- Avoid hard waits
- Use POM
- Use reusable utilities
- Use fixtures properly
- Keep tests independent
- Use environment configs

56. Common Anti-Patterns

Bad Practice

```
await page.waitForTimeout(10000);
```

Why bad?

- Slows execution
- Creates flaky tests

Preferred:

```
await expect(locator).toBeVisible();
```

57. Advanced Concepts

- Network mocking
- API chaining
- Parallel execution
- Multi-user sessions
- Visual testing
- Trace debugging
- Browser contexts
- Mobile emulation
- Accessibility testing

58. Mobile Emulation

```
const { devices } = require('@playwright/test');

use: {
  ...devices['iPhone 13']
}
```

59. Accessibility Testing

Playwright supports accessibility snapshots.

```
const snapshot = await page.accessibility.snapshot();
```

60. Most Important Topics to Revise

- Locators
- Auto waits
- Assertions
- Browser contexts
- POM
- Fixtures
- Hooks
- Parallel execution
- API testing
- Network interception
- Reporting
- Trace viewer
- CI/CD integration
- Playwright architecture

61. Final Interview Tips

- Explain concepts with practical examples.
 - Focus heavily on architecture and framework design.
 - Compare Playwright with Selenium.
 - Understand auto-waiting deeply.
 - Explain real-time challenges and solutions.
 - Be confident explaining POM and fixtures.
 - Revise API testing basics.
 - Practice writing locators and assertions.
-

62. Frequently Asked Coding Programs

Validate Page Title

```
const { test, expect } = require('@playwright/test');

test('Validate Title', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

Login Automation

```
const { test } = require('@playwright/test');

test('Login Test', async ({ page }) => {
  await page.goto('https://example.com/login');

  await page.fill('#username', 'admin');
  await page.fill('#password', 'admin123');
  await page.click('#loginBtn');
});
```

Handle Dropdown

```
await page.selectOption('#country', 'India');
```

Capture Screenshot on Failure

```
test.afterEach(async ({ page }, testInfo) => {
  if (testInfo.status !== testInfo.expectedStatus) {
    await page.screenshot({ path: 'failure.png' });
  }
});
```

API GET Request

```
const response = await request.get('https://reqres.in/api/users');  
console.log(await response.json());
```

63. Sample Enterprise Framework Structure

```
PlaywrightFramework  
├── tests  
├── pages  
├── fixtures  
├── utils  
├── test-data  
├── config  
├── reports  
├── screenshots  
├── playwright.config.js  
└── package.json
```

64. End-to-End Flow in Playwright Framework

```
Feature/Test File  
  ↓  
Page Object  
  ↓  
Playwright APIs  
  ↓  
Browser Context  
  ↓  
Browser Engine  
  ↓  
Application Under Test  
  ↓  
Assertions & Reports
```

End of Document